

SetInterval function basics in JavaScript

In this tutorial, we will learn how to work with timers in JavaScript. Having dealt with the timers, you can automatically perform any operations on the page after a specified period of time. For example, it will be possible to make an image slider in which the images will change every second.

To work with timers in JavaScript, the `setInterval` function is used, which runs the given code at certain intervals.

This function works as follows: as the first parameter, it takes `function source code`, and as the second parameter, the interval after which this function will be automatically called. The second parameter is in milliseconds (1000 milliseconds = 1 second).

Let's learn how the function works with an example. For example, let's write a code that every second will output something to the console.

First, let's make a function that outputs something to the console:

```
1 function timer() {  
2   console.log('!');  
3 }
```

And now with the help of `setInterval` we will make the function we created will be executed every second:

```
1 setInterval(timer, 1000);  
2  
3 function timer() {  
4   console.log('!');  
5 }
```

It is not necessary to create a separate function - you can simply pass an anonymous function to the first parameter of `setInterval`, like this:

```
1 setInterval(function() {  
2   console.log('!');  
3 }, 1000);
```

№ 1

©jsPmTmrLn

Start a timer that every 3 seconds will print something to the console.

Counter using setInterval function in JavaScript

Of course, it's not very interesting that our timer outputs the same thing every time. Let's complicate our task and make it so that every second the numbers are displayed in the console in ascending order: first 1, then 2, then 3 and so on.

To do this, we need a counter variable that will store its values between function runs. It's easy to see that you can just make a global variable:

```
1 let i = 0; // the global variable  
2  
3 setInterval(function() {  
4   i++;  
5   console.log(i);  
6 }, 1000);
```

Let's rewrite it more compactly:

```
1 let i = 0;  
2 setInterval(function() {  
3   console.log(++i);  
4 }, 1000);
```

Or even more compactly using an arrow function:

```
1 | let i = 0;
2 | setInterval(() => console.log(++i), 1000);
```

No 1

©jsPmTmrCn

Let a variable be given that initially stores the number 100. Start a timer that every second will decrease the value of this variable by 1 and print this value to the console.

Stopping a timer in JavaScript

You already know how to start a timer, now let's learn how to stop it. To do this, you need to know that each timer started with the `setInterval` function has a unique number. This number is returned by the `setInterval` function when the timer starts:

```
1 | let timerId = setInterval(function() {
2 |   console.log('!')
3 | }, 1000);
4 |
5 | alert(timerId); // will display the number of the timer
```

To stop the timer, use the `clearInterval` function, which takes a unique number of the timer to be stopped.

For example, let's start a timer that prints numbers to the console in ascending order, starting with 1. Let's stop the timer as soon as the number 10 is displayed on the screen:

```
1 | let i = 0;
2 |
3 | let timerId = setInterval(function() {
4 |   console.log(++i);
5 |
6 |   if (i >= 10) {
7 |     clearInterval(timerId);
8 |   }
9 | }, 1000);
```

No 1

©jsPmTmrTS

Let a variable be given that initially stores the number 10. Start a timer that every second will decrease the value of this variable by 1 and print this value to the console. As soon as the value of the variable reaches zero, stop the timer.

Button for starting a timer in JavaScript

In the previous lessons, our timer started its work as soon as the page loaded. Let's now make a button, by clicking on which our timer will start:

```
1 | <button id="start">start</button>
```

Let's write the appropriate JavaScript:

```
1 | let start = document.querySelector('#start');
2 |
3 | start.addEventListener('click', function() {
4 |   let i = 0;
5 |
6 |   setInterval(function() {
7 |     console.log(++i);
8 |   }, 1000);
9 | });
```

Make a button, by clicking on which a countdown will be displayed in the console, starting from 100.

Multiple pressing of the button for starting a timer in JavaScript

The code I provided above actually has some problem. This problem manifests itself if our button is clicked several times. In this case, *each click will start a new timer*.

That is, for example, three clicks on the button will cause three timers to be launched, and each of these timers will output the value of the counter to the console. This means that the values will change three times faster!

To solve the described problem, you just need to make sure that pressing the button again does not lead to a new start of the timer. For example, after the start of the timer, you can unbind the attached click handler from the button:

```
1 let start = document.querySelector('#start');
2
3 start.addEventListener('click', function func() {
4   let i = 0;
5
6   setInterval(function() {
7     console.log(++i);
8   }, 1000);
9
10  this.removeEventListener('click', func); // unbinds the handler
11 });
```

Take your solution to the previous problem. Check that pressing the button repeatedly speeds up the countdown. Fix this issue.

Buttons to start and stop a timer in JavaScript

Let's now make two buttons: by clicking on the first one, let the timer start, and by clicking on the second one, it will stop. Here everything will not be so simple and a trap awaits us. In order to understand the essence of this trick, I will describe the creation of the code in steps.

So, we make two buttons:

```
1 <button id="start">start</button>
2 <button id="stop">stop</button>
```

We get references to these buttons into variables:

```
1 let start = document.querySelector('#start');
2 let stop = document.querySelector('#stop');
```

By pressing the first button, we start the timer by writing its number to a variable:

```
1 start.addEventListener('click', function() {
2   let timerId = setInterval(function() {
3     console.log('!')
4   }, 1000);
5 });
```

And now we'll stop the timer by pressing the second button:

```
1 start.addEventListener('click', function() {
2   let timerId = setInterval(function() {
3     console.log('!')
4   }, 1000);
5 });
6
```

```

7 // Stopping the timer:
8 stop.addEventListener('click', function() {
9   clearInterval(timerId);
10 });

```

however, if we try to run the above code, we are in for a surprise: when we try to stop the timer, we find that the variable `timerId` is equal to `undefined`! Why did it happen? Therefore, when starting the timer, we made our variable `timerId` local inside the function bound to the `start` button.

To solve the problem, let's make the `timerId` variable global - in this case, it will be available both in the timer start function and in the stop function:

```

1 let timerId; // makes the variable global
2
3 start.addEventListener('click', function() {
4   timerId = setInterval(function() {
5     console.log('!!')
6   }, 1000);
7 });
8
9 stop.addEventListener('click', function() {
10  clearInterval(timerId);
11 });

```

No 1

©jsPmTmrSB

Let a variable be given that initially stores the number 100. There are also two buttons. By pressing the first button, start the timer, which every second will decrease the value of the variable by 1 and output the new value to the console. As soon as the value of the variable reaches zero, stop the timer.

Stop the timer by pressing the second button. Also stop the timer if the second button was not pressed, but the value of the variable reached zero.

No 2

©jsPmTmrSB

Some programmer wrote a code that, when a button is pressed, starts a timer that prints the current time to the console:

```

1 <input type="submit" id="start" value="start">
2 <input type="submit" id="stop" value="stop">

1 let start = document.querySelector('#start');
2 let stop = document.querySelector('#stop');
3
4 start.addEventListener('click', function() {
5   let timerId = setInterval(function() {
6     let date = new Date;
7     console.log(date.getMinutes() + ' ' + date.getSeconds());
8   }, 1000);
9 });
10
11 stop.addEventListener('click', function() {
12   clearInterval(timerId);
13 });

```

After running the code, however, it turned out that the stop button didn't work. Please fix the code author's mistake.

№3

©jsPmTmrSB

Another programmer also wrote code to solve the previous problem:

```
1 <input type="submit" id="start" value="start">
2 <input type="submit" id="stop" value="stop">

1 let start = document.querySelector('#start');
2 let stop = document.querySelector('#stop');
3
4 let timerId;
5
6 start.addEventListener('click', function() {
7   let timerId = setInterval(function() {
8     let date = new Date;
9     console.log(date.getMinutes() + ' ' + date.getSeconds());
10  }, 1000);
11 });
12
13 stop.addEventListener('click', function() {
14   clearInterval(timerId);
15 });
```

After running the code, however, again it turned out that the stop button did not work. Fix the code author's mistake.

№4

©jsPmTmrSB

Another programmer also wrote code to solve the previous problem:

```
1 <input type="submit" id="start" value="start">
2 <input type="submit" id="stop" value="stop">

1 let start = document.querySelector('start');
2 let stop = document.querySelector('stop');
3
4 let timerId;
5
6 start.addEventListener('click', function() {
7   setInterval(function() {
8     let date = new Date;
9     console.log(date.getMinutes() + ' ' + date.getSeconds());
10  }, 1000);
11 });
12
13 stop.addEventListener('click', function() {
14   clearInterval(timerId);
15 });
```

After running the code, however, again it turned out that the stop button did not work. Fix the code author's mistake.

№5

©jsPmTmrSB

Another programmer also wrote code to solve the previous problem:


```

1 <input type="submit" id="start" value="start">
2 <input type="submit" id="stop" value="stop">

1 let start = document.querySelector('start');
2 let stop = document.querySelector('stop');
3
4 let timerId;
5
6 start.addEventListener('click', function() {
7   let timerId = setInterval(function() {
8     let date = new Date;
9     console.log(date.getMinutes() + ' ' + date.getSeconds());
10  }, 1000);
11 });
12
13 stop.addEventListener('click', function() {
14   clearInterval();
15 });

```

After running the code, however, again it turned out that the stop button did not work. Fix the code author's mistake.

№6

©jsPmTmrSB

The code I gave in the theoretical part does not take into account the fact that several clicks can be made on the start button. To fix this problem, you can unbind the event from this button by pressing the start button, and bind it back by pressing the stop button. Fix the problem.

Practice on timers and working with the DOM in JavaScript

Let us have an input:

```
1 <input id="elem" value="1">
```

Let's make a timer that every second increments the value of the `value` attribute of our input by one:

```

1 let elem = document.querySelector('#elem');
2
3 setInterval(function() {
4   elem.value = Number(elem.value) + 1;
5 }, 1000);

```

№1

©jsPmTmrDm

Explain why the `Number` function is used in the above code. What happens if you don't write this function in this code?

№2

©jsPmTmrDm

Let the input in the `value` attribute initially contain the number 10. Start a timer that will decrease this number by one every second.

№3

©jsPmTmrDm

Modify the previous task so that as soon as the content of the input becomes zero, the timer stops running.

Timers and context loss in JavaScript

When using timers in event handlers, we are faced with problems with loss of context. Let's look at an example.

Let us have an input:

```
1 | <input id="elem" value="text">
```

Let an anonymous function be triggered by clicking on this input and a timer will start inside this function, outputting something to the console every second:

```
1 | let elem = document.querySelector('#elem');
2 |
3 | elem.addEventListener('click', function() {
4 |   setInterval(function() {
5 |     console.log('!!!'); // outputs something to the console
6 |   }, 1000);
7 | });
```

As long as everything is working correctly. But now let's say we want to output `value` of our input to the console - a surprise awaits us: `undefined` will be output to the console:

```
1 | elem.addEventListener('click', function() {
2 |   setInterval(function() {
3 |     console.log(this.value); // will output undefined
4 |   }, 1000);
5 | });
```

The thing is that we get a function within a function: there is an external anonymous function that is called on click and an internal anonymous function that the timer starts. In the outer function, `this` points to an input, but not in the inner one. There is a loss of context.

Why is `undefined` displayed instead of throwing an error in the console, as it was in previous lessons? Because `this` inside a function called via `setInterval` points to `window`.

This means that we are trying to read the `value` property of the `window` object, like this: `window.value`, but there is no such property in it, and we get `undefined` (not an error).

Let's fix the problem by introducing `self`:

```
1 | elem.addEventListener('click', function() {
2 |   let self = this;
3 |
4 |   setInterval(function() {
5 |     console.log(self.value);
6 |   }, 1000);
7 | });
```

№ 1

©jsPmTmrTCL

Let this code be given:

```
1 | <input type="button" id="elem" value="1">

1 | let elem = document.querySelector('#elem');
2 |
3 | elem.addEventListener('click', function() {
4 |   setInterval(function() {
5 |     this.value = Number(elem.value) + 1;
6 |   }, 1000);
7 | });
```

The author of the code wanted that when a button is pressed, the value of this button increases by 1 every second. However, when the button is pressed, nothing happens at all. Please correct the code author's mistake. Write a text in which you will explain to the author of the code why this error occurred.

Practice on timers and working with the DOM in JavaScript

№ 1

ⓧjsPmTmrPr

Given a button. Given a paragraph whose text is a number. By clicking on the button, start a timer that every second will increase the text of the paragraph by 1.

№ 2

ⓧjsPmTmrPr

Given a button. Given a paragraph whose text is a number, for example, 10. By pressing the button, start the timer, which every second will decrease the text of the paragraph by 1. As soon as the value of the paragraph becomes zero - stop the timer.

№ 3

ⓧjsPmTmrPr

Given an input with a number. Make it so that every second the square of the number that is written in it appears in it.

№ 4

ⓧjsPmTmrPr

Given an input. A number is entered into it. When focus is lost, set the paragraph below to start counting down from the number entered. When the countdown reaches zero, it must end.

№ 5

ⓧjsPmTmrPr

Given an input, a button, and a paragraph. A number is entered into the input. By pressing the button, write the entered number into the text of the paragraph and start the countdown in the paragraph: let the number in the paragraph decrease by one every second until it reaches zero.

№ 6

ⓧjsPmTmrPr

Given a paragraph and two buttons. Make it so that when you click on the first button in the paragraph, the timer starts ticking from 1 to infinity, and when you click on the second button, the timer stops.

№ 7

ⓧjsPmTmrPr

Given a paragraph. Make it so that every second it changes its color from red to green and vice versa.

№ 8

ⓧjsPmTmrPr

If you display the current moment of time on the screen every second, you can make a ticking clock. Implement the same clock as shown in the sample below:

00:32:52

Delay before execution in JavaScript

Let's now look at the `setTimeout` function, which allows you to make a delay before running the code. This delay, unlike the `setInterval` function, will happen only once.

The first parameter of `setTimeout` takes the source code of the function, and the second parameter is the delay in milliseconds before starting this function.

Let's see how the function works on some example. Let's say we have a button like this:


```
1 <input type="submit" id="elem">
```

Get a reference to our button into a variable:

```
1 let elem = document.querySelector('#elem');
```

Let's now make it so that when the button is pressed, `alert` is displayed on the screen, but not immediately, but after 3 seconds after pressing:

```
1 elem.addEventListener('click', function() {  
2   setTimeout(function() {  
3     alert('!');  
4   }, 3000);  
5 });
```

No 1

⊗jsPmTmrDI

Given a paragraph. Write code that will display a message in this paragraph 10 seconds after the page loads.

Timer start in JavaScript

Although the `setTimeout` function is not designed to create timers, you can still make them with it if you use recursion:

Working with recursion in JavaScript

In programming, there is such a concept as *recursion* - this is when a function calls itself. Let's look at an example. Using recursion, we derive numbers from 1 to 10:

```
1 let i = 1;  
2  
3 function func(){  
4   console.log(i);  
5   i++;  
6  
7   if (i <= 10){  
8     func(); // here the function calls itself  
9   }  
10 }  
11 func();
```

Let's discuss how this code works.

We have the global variable `i` and the function `func`, inside which the contents of the variable `i` are displayed to the console, and then `++` is performed.

If our variable `i` is less than or equal to 10, then the function is called again. Since the variable `i` is global, then with each new call to the function it will contain the value of the variable `i` specified in the previous call.

It turns out that the function will call itself until `i` becomes greater than 10.

Please note that in our case it is impossible to run the function without `if` - if you do this, you will get an endless functions call.

Timer start in JavaScript

Although the `setTimeout` function is not designed to create timers, you can still make them with it if you use recursion:

```
1 let i = 0;
2
3 function timer() {
4   setTimeout(function() {
5     console.log(++i);
6
7     timer(); // let's call ourselves
8   }, 1000);
9 }
10 timer();
```

You can stop such a timer simply without letting the recursion happen:

```
1 let i = 0;
2
3 function timer() {
4   setTimeout(function() {
5     console.log(++i);
6
7     if (i < 10) { // runs only if counter is less than 10
8       timer();
9     }
10   }, 1000);
11 }
12 timer();
```

№ 1

©jsPmTmrDT

Print the number 0 to the console. After one second - print the number 1, after two seconds - print the number 2, after 3 seconds - print the number 3. And so on ad infinitum.